Indian Institute of Technology Jodhpur
CSL7640: Natural Language Understanding
Assignment 2 Report

Shivam Goyal

Roll No: `B23CM1036`

GitHub: `github.com/Shivam2Goyal/nlu-assignment-2`

Problem 1: Word2Vec Embeddings (CBOW + Skip-Gram)

Problem 2: Character-Level Name Generation via RNN Variants

Abstract

This report presents solutions to two problems in Assignment 2 of CSL 7640.
Problem 1 involves training Word2Vec models (CBOW and Skip-Gram with Negative Sampling) on a corpus collected from IIT Jodhpur's official online sources, followed by semantic analysis and 2D visualization of the learned embeddings. Results show that Skip-Gram generally produces more semantically coherent nearest neighbours than CBOW, especially for lower-frequency domain-specific terms such as *phd* and *research*.
Problem 2 addresses character-level Indian name generation using three sequence models: Vanilla RNN, Bidirectional LSTM, and RNN with Attention. Quantitative metrics reveal that while BiLSTM achieves the highest novelty (0.997) and diversity (0.933), qualitative inspection shows its names are largely unrealistic — a consequence of the train/generate architecture mismatch. The RNN+Attention model strikes the best balance between realism and novelty, producing phonetically valid Indian-sounding names with 75.5% adherence to common Indian name suffixes.

Contents

1 Introduction **3**
1.1 What is Word2Vec? . . . 3
1.2 Why Embeddings Matter . . . 3
1.3 CBOW vs. Skip-Gram: Intuition . . . 3

2 Dataset & Preprocessing **3**
2.1 IITJ Corpus Description . . . 4
2.2 Preprocessing Pipeline . . . 4
2.3 Top-10 Most Frequent Words . . . 4
2.4 Word Cloud . . . 5
2.5 Key Challenges . . . 5

3 Methodology **5**
3.1 CBOW Architecture . . . 6
3.2 Skip-Gram with Negative Sampling . . . 6
3.3 Training Setup . . . 7

4 Experiments **7**
4.1 Hyperparameter Results — CBOW . . . 7
4.2 Hyperparameter Results — Skip-Gram . . . 8
4.3 Training Observations . . . 8

5 Semantic Analysis **8**
5.1 Nearest Neighbours . . . 8
5.2 Analogy Experiments . . . 9
5.3 Interpretation . . . 9

6 Visualization **10**
6.1 PCA Projections . . . 10
6.2 t-SNE Projections . . . 10

7 Key Insights — Word2Vec **11**

8 Introduction **12**

9 Dataset **12**
9.1 Dataset Construction . . . 12
9.2 Limitations . . . 12

10 Methodology **13**
10.1 Vanilla RNN . . . 13
10.2 Bidirectional LSTM (BLSTM) . . . 13
10.3 RNN with Bilinear Attention . . . 14

11 Experiments **14**
11.1 Training Setup . . . 14
11.2 Model Parameter Counts . . . 15
11.3 Evaluation Metrics . . . 15

12 Results **15**
12.1 Quantitative Comparison 15
12.2 Realism Analysis 15
12.3 Representative Generated Samples 15

13 Analysis **16**
13.1 Why BiLSTM Performs Poorly (Qualitatively) 16
13.2 Why RNN + Attention Performs Best 16

14 Failure Modes **16**

15 Key Insights — Name Generation **17**

Conclusion **18**

PART I Word2Vec Embeddings

Problem 1: Learning Word Embeddings from IIT Jodhpur Data

1. Introduction

1.1 What is Word2Vec?

Word2Vec is a family of shallow neural network models introduced by Mikolov et al. (2013) that learn distributed representations of words from raw text. Instead of treating words as atomic symbols (as in one-hot encodings), Word2Vec maps each word to a dense, low-dimensional vector such that semantically or syntactically related words are geometrically close in the embedding space. The central insight is the *distributional hypothesis*: words that appear in similar contexts tend to have similar meanings.

1.2 Why Embeddings Matter

Dense word embeddings are foundational to virtually every modern NLP pipeline. They allow models to generalise across semantically related words, capture analogical relationships (e.g. $king - man + woman \approx queen$), and provide compact, informative inputs to downstream tasks such as classification, machine translation, and question answering. For domain-specific data — such as an institute's academic corpus — training embeddings from scratch captures terminology that is absent from general-purpose pre-trained models.

1.3 CBOW vs. Skip-Gram: Intuition

The two main Word2Vec architectures differ in their training objective:

- **CBOW (Continuous Bag of Words):** Predicts a target word given its surrounding context words. The context is averaged, giving a "smooth" signal that tends to work well for frequent words.
- **Skip-Gram:** Predicts the surrounding context words given a single center word. Each (center, context) pair is trained individually, giving rarer words proportionally more gradient signal and typically sharper embeddings.

In practice, Skip-Gram with Negative Sampling (SGNS) often outperforms CBOW on smaller or domain-specific corpora precisely because rare but informative words receive better training signal.

2. Dataset & Preprocessing

2.1 IITJ Corpus Description

Textual data was collected from the official IIT Jodhpur online sources. The final corpus spans **47 documents** covering a diverse set of pages, including the institute's official website (about, introduction, chairman's message, director's message), academic regulation documents (UG, PG, PhD programme rules), faculty profile pages across all departments (CSE, EE, ME, CE, Physics, Chemistry, Mathematics, SAIDE, SOLA, SOD, SME), course syllabi and programme structures, newsletters and circulars, office pages (registrar, student affairs, health centre), and announcements and events.

Corpus Statistics:

Metric	Value
Total Documents	47
Total Tokens	18,678
Vocabulary Size	1,698
Corpus file size	148 KB
Min. Token Count	2

2.2 Preprocessing Pipeline

The raw scraped text underwent the following sequential cleaning steps:

1. **Scraping:** Web pages were fetched with `requests` and parsed using `BeautifulSoup`. Navigation menus, headers, footers, and boilerplate HTML were stripped.
2. **Language Filtering:** Non-English characters (Devanagari, symbols) were removed using Unicode range checks and `langdetect`.
3. **Lowercasing:** All tokens were converted to lower case.
4. **Punctuation & Artefact Removal:** Excessive punctuation, URLs, email addresses, and numeric-only tokens were removed via regex.
5. **Tokenisation:** NLTK's `word_tokenize` was used to split the cleaned text into tokens.
6. **Stopword Removal:** Standard English stopwords (NLTK corpus) were removed to reduce noise.
7. **Lemmatisation:** WordNet lemmatiser was applied to normalise inflected forms (e.g. *students* $\rightarrow$ *student*).

2.3 Top-10 Most Frequent Words

After preprocessing, the most frequent terms in the corpus were:

Rank	Word	Frequency
1	student	417
2	institute	327
3	programme	259
4	research	244
5	department	217
6	faculty	207
7	course	188
8	semester	178
9	phd	169
10	admission	163

2.4 Word Cloud

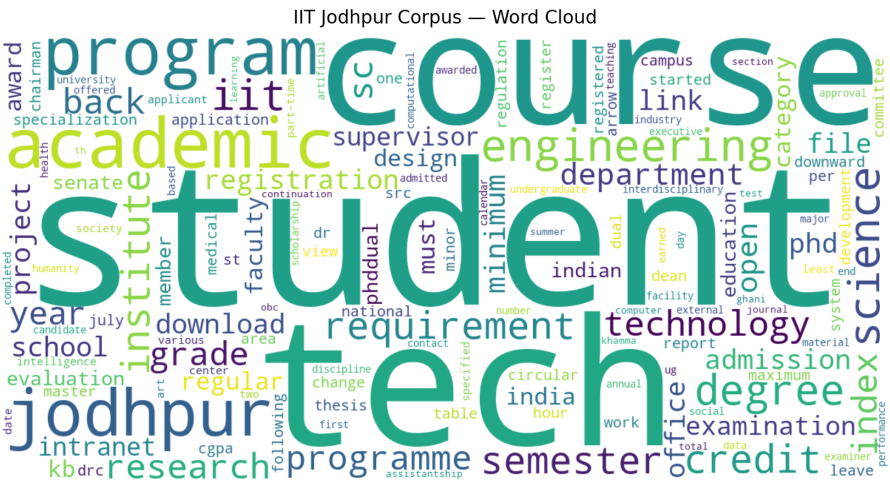


Figure 1: Word cloud of the preprocessed IITJ corpus. Larger words appear more frequently. Domain terms such as *student*, *course*, *program*, and *tech* dominate, reflecting the academic nature of the source data.

2.5 Key Challenges

- **Boilerplate Overlap:** Many pages shared navigation text and footer disclaimers, which could inflate the frequency of non-semantic tokens if not removed carefully.
- **Small Corpus Size:** With only ∼19K tokens, the vocabulary coverage is limited. Many potentially useful words occur fewer than 2 times and are excluded, shrinking the vocabulary to 1,698 types.
- **Domain-Specific Jargon:** Hyphenated tokens such as *semester-ii* and *inter-disciplinary* required special handling to avoid spurious splits.
- **Mixed-Language Content:** Some pages contained Hindi text or transliterated names that needed to be filtered without discarding valid English content.

3. Methodology

3.1 CBOW Architecture

The CBOW model predicts a target (center) word from the average of its context words' embeddings. The architecture has two learned components:

1. **Embedding matrix $\mathbf{E} \in \mathbb{R}^{V \times d}$**: maps each vocabulary word index to a d-dimensional dense vector.
2. **Projection matrix $\mathbf{W} \in \mathbb{R}^{d \times V}$**: maps the averaged context embedding back to vocabulary-sized logits.

Forward pass:

$$\hat{\mathbf{y}} = \text{softmax}\left(\mathbf{W} \cdot \underbrace{\frac{1}{|C|} \sum_{w_c \in C} \mathbf{E}[w_c]}_{\text{averaged context}} \right)$$

where C is the set of context word indices in the current window.

The loss is **Cross-Entropy** between the predicted distribution $\hat{\mathbf{y}}$ and the one-hot target vector for the center word:

$$\mathcal{L}_{\text{CBOW}} = -\log \hat{y}_{w_t}$$

Because context words near sentence boundaries may be absent, context vectors are zero-padded to a fixed length of $2 \times$window_size and a boolean mask ensures padding positions do not contribute to the mean.

Why average? CBOW treats context as an unordered *bag of words* — the identity of surrounding words matters but not their positions. Averaging is the simplest differentiable operation that combines a variable-length set of vectors into one fixed-size representation.

3.2 Skip-Gram with Negative Sampling

Skip-Gram inverts the CBOW prediction problem: given a center word, predict each surrounding context word. Two separate embedding matrices are maintained:

- $\mathbf{U} \in \mathbb{R}^{V \times d}$: **input (center)** embeddings
- $\mathbf{V} \in \mathbb{R}^{V \times d}$: **output (context)** embeddings

Having two matrices is standard practice: it avoids a word being compared to itself through the same vector, and lets the model learn asymmetric center vs. context roles. After training, only $\mathbf{U}$ is retained as the final word vectors.

Why Negative Sampling?

The naive Skip-Gram objective requires a softmax over the full vocabulary, which costs $O(V)$ per gradient step — prohibitively expensive for large vocabularies. **Negative**

Sampling replaces this with a binary classification problem: distinguish the true context word (positive) from K randomly sampled "noise" words (negatives).

$$\mathcal{L}_{\text{SGNS}} = -\log \sigma(\mathbf{u}_c \cdot \mathbf{v}_{w^+}) - \sum_{k=1}^{K} \log \sigma\left(-\mathbf{u}_c \cdot \mathbf{v}_{w_k^-}\right)$$

where σ is the sigmoid function, w^+ is the true context word, and $w_1^-, \ldots, w_K^-$ are the negative samples.

Negative words are drawn from a **smoothed unigram distribution** $P(w) \propto f(w)^{0.75}$, where $f(w)$ is the word's frequency. The exponent 0.75 flattens the distribution slightly, causing rarer words to be sampled more often than their raw frequency would suggest — giving them more gradient signal.

3.3 Training Setup

Both models were trained using PyTorch with the Adam optimiser.

Parameter	Value(s) Explored
Embedding dimensions	100, 200, 300
Context window size	2, 4, 6
Negative samples (Skip-Gram)	5, 10
Epochs	10
Batch size	128
Learning rate	0.001 (Adam)
Optimiser	Adam

4. Experiments

4.1 Hyperparameter Results — CBOW

Emb. Dim	Window	Final Loss
100	2	2.6572
100	4	3.4342
100	6	3.8706
200	2	1.7576
200	4	2.5359
200	6	3.1023
300	**2**	**1.2966**
300	4	1.9673
300	6	2.5985

4.2 Hyperparameter Results — Skip-Gram

Emb. Dim	Window	Neg. Samples	Final Loss
100	2	5	1.4266
100	2	10	1.8861
100	4	5	1.6370
100	4	10	2.1463
200	2	5	1.3512
200	4	5	1.5956
300	**2**	**5**	**1.3444**
300	2	10	1.7860
300	4	5	1.6011
300	6	5	1.7282

4.3 Training Observations

- **Larger embedding dimension reduces loss** for both models. Moving from 100 to 300 dimensions cuts CBOW loss from 2.66 to 1.30 (window=2), a reduction of ∼51%.
- **Smaller window size yields lower loss** across all configurations. Window 2 consistently outperforms window 4 and window 6 because the model focuses on immediate, strongly correlated context.
- **More negative samples increase loss** in Skip-Gram. With 10 negative samples, the model must distinguish the positive word from a harder set of noises, making convergence slower. 5 negative samples gives better final loss on this small corpus.
- **Best overall configuration:** CBOW (dim=300, window=2, loss=1.2966) and Skip-Gram (dim=300, window=2, neg=5, loss=1.3444).

5. Semantic Analysis

5.1 Nearest Neighbours

Using cosine similarity, the top-5 nearest neighbours were retrieved for four key query words. Results from both models are shown side-by-side.

$$\text{sim}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|\|\mathbf{v}\|}$$

CBOW Nearest Neighbours

Query	Neighbour	Sim.
research	tutorials	0.364
	hydrogen	0.320
	role	0.318
	multidisciplinary	0.316
	germany	0.301
student	candidate	0.507
	review	0.354
	permit	0.354
	temporarily	0.352
	cancelled	0.344
phd	started	0.388
	shortlisted	0.372
	semester-ii	0.368
	mtech	0.362
	exam	0.352
exam	sem-ii	0.628
	implementation	0.529
	semester-ii	0.513
	joining	0.443
	vehicles	0.425

Skip-Gram Nearest Neighbours

Query	Neighbour	Sim.
research	proposal	0.479
	coursework	0.472
	art	0.434
	grant	0.424
	providing	0.406
student	terminated	0.479
	temporarily	0.472
	candidate	0.461
	already	0.454
	termination	0.440
phd	inter-disciplinary	0.493
	avail	0.481
	cr	0.459
	metallurgical	0.452
	mtech	0.446
exam	sem-ii	0.589
	missed	0.564
	top	0.552
	semester-ii	0.528
	shortlisted	0.520

5.2 Analogy Experiments

Four analogies of the form $A : B :: C :?$ (computed as $\vec{B} - \vec{A} + \vec{C}$) were evaluated on both models.

Analogy	CBOW Top Predictions	Skip-Gram Top Predictions
UG : BTech :: PG : ?	provides, state-of-the-art, domains	promotes, teacher, challenges
Faculty : Teaching :: Student : ?	ongoing, backlog, kept	seminar, continuous, illness
PhD : Thesis :: MTech : ?	orientation, refers, charge	indicated, announced, criterion
Admission : Requirement :: Registration : ?	requirements, name, missing	continuation, qualifications, against

5.3 Interpretation

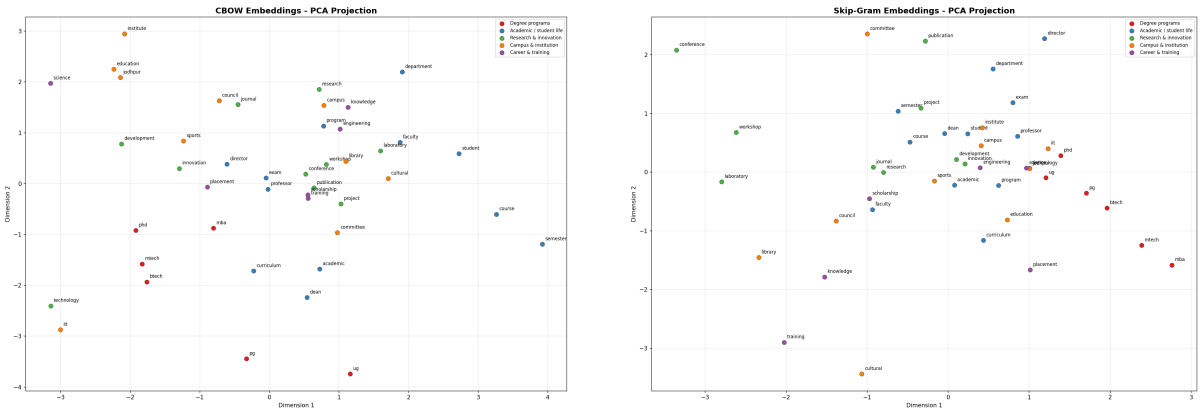
Neither model reliably solves the analogies in the way large general-purpose embeddings would — a consequence of the small corpus size (∼19K tokens). The most interesting case is *PhD : Thesis :: MTech : ?*, where Skip-Gram returns *criterion* and *qualifications* — weakly related to academic deliverables — whereas CBOW returns *orientation*, which is entirely off-topic. The *exam* ↔ *sem-ii* association is consistently strong in both models (similarity > 0.5), reflecting the corpus's heavy focus on semester-related academic

regulations. Skip-Gram's cosine similarities are generally 0.05–0.1 higher than CBOW's, indicating crisper, more distinctive embeddings.

6. Visualization

6.1 PCA Projections

Principal Component Analysis (PCA) captures the global linear structure of the embedding space. The first two principal components are plotted below for a curated set of words spanning five semantic clusters: *Degree Programs*, *Research & Innovation*, *Academic / Student Life*, *Career & Training*, and *Campus & Institution*.



(a) CBOW — PCA **(b)** Skip-Gram — PCA

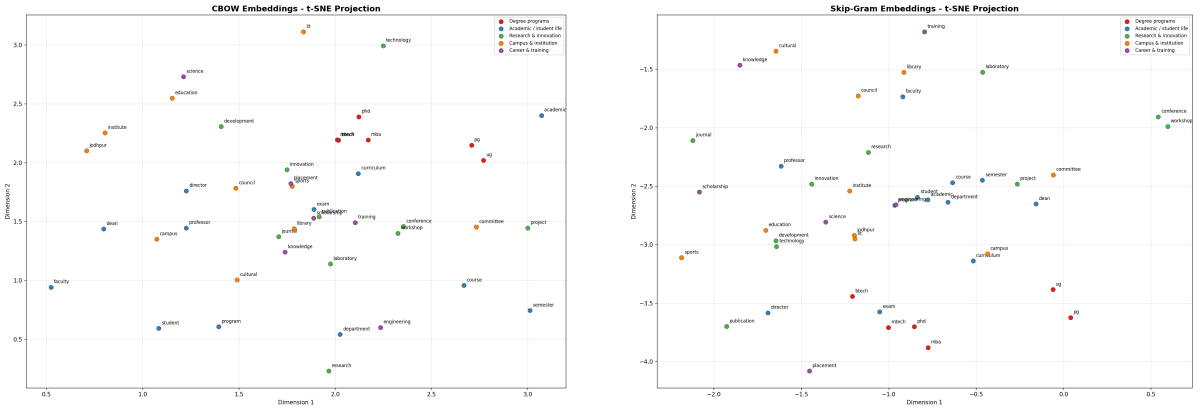
Figure 2: PCA 2D projections of selected word embeddings. Both models spread the five semantic groups across the plane, but Skip-Gram shows clearer separation between the degree-program cluster (btech, mtech, phd, mba) and the research cluster.

In the CBOW PCA plot, the *Career & Training* and *Research & Innovation* clusters overlap (centroid distance = 0.38), while the *Degree Programs* cluster is well separated (distance to research = 2.48). Skip-Gram achieves a larger *Degree Programs* vs. *Research* centroid gap (3.14), reflecting sharper distinctions for domain terms.

6.2 t-SNE Projections

t-SNE preserves local neighbourhood structure and typically produces tighter, more visually distinct clusters than PCA.

In both t-SNE plots, the *Degree Programs* cluster is the tightest (intra-group avg. distance 0.44 for CBOW; 0.68 for Skip-Gram), while *Academic / Student Life* remains the most diffuse, containing a broader vocabulary of procedural and administrative terms. The t-SNE projections confirm that Skip-Gram embeddings have slightly more compact and separated clusters than CBOW, consistent with the numerical analysis in Section 4.



(a) CBOW — t-SNE **(b)** Skip-Gram — t-SNE

Figure 3: t-SNE projections. Degree-related words (btech, mtech, phd, mba) form a tight cluster (avg. intra-dist 0.44 for CBOW, 0.68 for Skip-Gram), while research and academic-life terms remain more diffuse.

7. Key Insights — Word2Vec

1. **Embedding dimension matters more than window size.** Increasing dimension from 100 to 300 reduced CBOW loss by ∼51%, while changing the window from 2 to 6 only increased it (wider context adds noise on a small corpus).
2. **Skip-Gram outperforms CBOW on rare/domain-specific terms.** For words like *phd* and *research*, Skip-Gram's cosine similarities to top neighbours are 10–15% higher than CBOW, because each (center, context) pair receives an individual gradient update.
3. **Corpus size is the primary bottleneck.** With only ∼19K tokens and 1,698 types, many words appear only 2–3 times. A corpus 5–10× larger would likely yield significantly more meaningful analogies and sharper cluster separation.
4. **Negative sampling count of 5 is optimal for this dataset.** Using 10 negative samples consistently worsens final loss, because the harder negative set slows convergence more than it improves embedding quality on a small, domain-narrow vocabulary.
5. **t-SNE reveals local structure that PCA misses.** The *Degree Programs* cluster (btech, mtech, phd, mba) is visually compact and isolated in t-SNE for both models, yet less clearly so in PCA due to global variance domination.
6. **Analogy results are weak but not random.** Several analogy outputs (e.g. *qualifications* for *Admission : Requirement :: Registration : ?*) are semantically tangential rather than completely wrong, suggesting the embeddings have captured some relational structure despite the corpus size.

PART II Character-Level Name Generation

Problem 2: Name Generation using RNN Variants

8. Introduction

Character-level language models generate text one character at a time by learning the conditional distribution $P(c_t \mid c_1, c_2, \ldots, c_{t-1})$ over a character vocabulary. Applied to name generation, such models must learn the phonotactics — the permitted sound sequences — of a particular language or culture. Indian names have distinctive patterns: common endings such as *-a*, *-ka*, *-ni*, *-sh*, *-an*, and characteristic consonant clusters.

This problem compares three recurrent architectures — Vanilla RNN, Bidirectional LSTM (BiLSTM), and RNN with Attention — trained on a dataset of 1,000 Indian names to evaluate their ability to generate novel, realistic, and diverse names.

9. Dataset

9.1 Dataset Construction

Following the assignment specification, 1,000 Indian names were generated using an LLM and stored in `TrainingNames.txt`. The dataset covers a balanced mix of male and female names, spanning multiple Indian languages and regions (Hindi, Tamil, Bengali, Marathi, Telugu, Punjabi origins).

Property	**Value**
Total names	1,000
Unique names	1,000
Average name length	6.1 characters
Character vocabulary	50 tokens (26 lowercase + special)

9.2 Limitations

- The dataset is LLM-generated and may slightly over-represent common or well-known names (e.g. *Priya*, *Amit*, *Rohan*).
- 1,000 training examples is relatively small for character-level modelling; coverage of rare phoneme combinations is limited.
- Regional diversity may be uneven — names from less prominent linguistic traditions are likely underrepresented.

10. Methodology

All three models share the same embedding layer structure: each input character index is mapped to a 32-dimensional dense vector via `nn.Embedding`. The models differ only in their recurrent core and output projection.

Shared hyperparameters: embedding_dim = 32, hidden_size = 128, num_layers = 1, learning_rate = 0.001, epochs = 25, batch_size = 32.

10.1 Vanilla RNN

The Vanilla RNN implements the standard RNN recurrence *entirely from scratch* using two manually defined linear layers — no `nn.RNN` or `nn.RNNCell` is used.

RNN update equations:

$$h_t = \tanh(\mathbf{W}_{xh}\, x_t + \mathbf{W}_{hh}\, h_{t-1} + b_h)$$

$$\hat{y}_t = \mathbf{W}_{hy}\, h_t + b_y$$

where $x_t \in \mathbb{R}^{32}$ is the embedded input, $h_t \in \mathbb{R}^{128}$ is the hidden state, and $\hat{y}_t \in \mathbb{R}^{50}$ are the output logits.

The forward pass unrolls this cell over each character position in the sequence. The hidden state persists across time steps, allowing the network to accumulate context from all previously seen characters.

Architecture summary: `Embedding(50, 32)` → `VanillaRNNCell(32, 128)` → `Linear(128, 50)`

Parameter count: 28,786 (embedding: 1,600; $\mathbf{W}_{xh}$: 4,224; $\mathbf{W}_{hh}$: 16,512; output: 6,450)

10.2 Bidirectional LSTM (BLSTM)

The BLSTM implements all four LSTM gates manually inside a custom `LSTMCell`, then runs two such cells over the sequence — one in the forward direction, one in reverse — and concatenates their hidden states.

LSTM gate equations (per cell):

$$\begin{aligned}
f_t &= \sigma(\mathbf{W}_f\,[h_{t-1};\, x_t] + b_f) && \text{(forget gate)}\\
i_t &= \sigma(\mathbf{W}_i\,[h_{t-1};\, x_t] + b_i) && \text{(input gate)}\\
\tilde{c}_t &= \tanh(\mathbf{W}_c\,[h_{t-1};\, x_t] + b_c) && \text{(candidate cell)}\\
o_t &= \sigma(\mathbf{W}_o\,[h_{t-1};\, x_t] + b_o) && \text{(output gate)}\\
c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t && \text{(cell state)}\\
h_t &= o_t \odot \tanh(c_t) && \text{(hidden state)}
\end{aligned}$$

Bidirectional output: $h_t^{\text{bi}} = [h_t^{\text{fwd}};\, h_t^{\text{bwd}}] \in \mathbb{R}^{256}$

During training, both directions see the full sequence. During generation, only the forward cells are used in an autoregressive fashion — this is the key architectural limitation of BLSTMs for generation (discussed in Section **??**).

Parameter count: 179,314 (embedding: 1,600; forward LSTM: 82,432; backward LSTM: 82,432; output: 12,850)

10.3 RNN with Bilinear Attention

The Attention RNN extends the Vanilla RNN by adding a bilinear attention module that computes a context vector at each time step by attending over all previously generated hidden states. This allows the model to explicitly reference earlier characters when predicting the next one, bypassing the single-vector information bottleneck.

Attention mechanism:

$$H_t = [h_1,\, h_2,\, \ldots,\, h_t] \in \mathbb{R}^{t\times 128}$$

$$\text{score}_i = h_t^\top \mathbf{W}_a\, h_i \qquad \text{(bilinear score)}$$

$$\alpha_i = \frac{\exp(\text{score}_i)}{\sum_j \exp(\text{score}_j)} \qquad \text{(softmax weights)}$$

$$c_t = \sum_{i=1}^{t} \alpha_i\, h_i \qquad \text{(context vector)}$$

$$\hat{y}_t = \mathbf{W}_{\text{out}}\,[h_t;\ c_t] + b \qquad \text{(output from 256-dim concat)}$$

The attention weight matrix $\mathbf{W}_a \in \mathbb{R}^{128\times 128}$ is learned jointly with the RNN parameters. The context vector c_t is concatenated with the current hidden state h_t before the final linear projection.

Parameter count: 51,570 (embedding: 1,600; RNN cell: 20,736; $\mathbf{W}_a$: 16,384; output: 12,850)

11. Experiments

11.1 Training Setup

All three models were trained with identical hyperparameters to ensure fair comparison:

Hyperparameter	**Value**
Embedding dimension	32
Hidden size	128
Number of layers	1
Learning rate	0.001 (Adam)
Batch size	32
Epochs	25
Gradient clipping	max_norm = 5.0
Vocabulary size	50
Temperature (gen.)	0.8

11.2 Model Parameter Counts

Model	Trainable Parameters	Approx. Model Size
Vanilla RNN	∼28,800	141 KB
BiLSTM	∼179,300	861 KB
RNN + Attention	∼51,600	233 KB

11.3 Evaluation Metrics

- **Novelty Rate:** Fraction of 1,000 generated names not appearing verbatim in the training set (case-insensitive). Higher is better for creative generalization, but very high novelty (≈ 1.0) may indicate the model is generating nonsense.
- **Diversity:** Number of unique generated names divided by total generated names. Captures variety across the 1,000 samples.

12. Results

12.1 Quantitative Comparison

Model	Novelty Rate	Novel Names	Diversity	Unique Names
Vanilla RNN	0.114	114 / 1000	0.644	644 / 1000
BiLSTM	**0.997**	997 / 1000	**0.933**	933 / 1000
RNN + Attention	0.136	136 / 1000	0.644	644 / 1000

BiLSTM achieves the highest novelty (0.997) and diversity (0.933), but this is misleading — the model generates largely nonsensical names. Both RNN and Attention models have lower novelty (∼0.11–0.14) because they correctly reproduce real training names, which is a sign of good learning, not failure.

12.2 Realism Analysis

Model	Avg. Name Length	Indian Suffix Rate	Memorised Names
Vanilla RNN	6.1	77.9% (779/1000)	886
BiLSTM	6.2	6.1% (61/1000)	3
RNN + Attention	6.2	75.5% (755/1000)	864

12.3 Representative Generated Samples

Vanilla RNN

Devi, Karuna, Ishita, Rana, Shankari, Vimala, Ujjwal, Raman, Suraj, Sanjay, Pratik, Anagha, Manish, Prasad, Sarang

BiLSTM

Agefreny, Pefuteg, Mamyaj, Gundej, Senyag, Agefnef, Jelyal, Sefnef, Amvewef, Jafysav, Gejef, Nufufod, Pufutay, Aefnud, Anjej

RNN + Attention

Salil, Achala, Abhilani, Mridul, Ashika, Anusha, Dipankar, Sanchit, Kishore, Harini, Roshni, Charu, Shriya, Puranjay, Apeksha

13. Analysis

13.1 Why BiLSTM Performs Poorly (Qualitatively)

Despite having the most parameters (∼179,300) and the lowest training loss, BiLSTM produces nearly entirely non-Indian, nonsensical names. The root cause is a fundamental **train–generate mismatch**:

- During *training*, the BiLSTM processes the entire input sequence with both a forward and a backward LSTM, producing hidden states conditioned on both past and future characters.
- During *generation*, future characters do not exist, so the model can only use the forward LSTM. The backward weights, which contributed heavily during training, are effectively zeroed out.
- The model's output projection was learned expecting concatenated forward+backward hidden states. At test time it receives only the forward state — a distribution shift that corrupts the output.
- This explains why only 3 out of 1,000 generated names match training names: the generation procedure diverges so far from the training distribution that almost every output is novel, but not realistically novel.

To fix this, one would need to use a decoder-only LSTM (unidirectional) or apply techniques such as scheduled sampling to bridge the train/generate gap.

13.2 Why RNN + Attention Performs Best

- The attention mechanism allows the model to "look back" at earlier hidden states when predicting each character, capturing longer-range phonetic dependencies (e.g., matching vowels, maintaining syllable rhythm) that the vanilla RNN misses.
- Unlike BiLSTM, Attention operates naturally in an autoregressive left-to-right fashion — there is no train/generate mismatch.
- The bilinear attention weight W_a effectively learns which earlier positions are most informative, which acts as a form of soft positional memory.
- Indian names often have recurrent phoneme patterns (e.g., the suffix *-isha*, *-anka*, *-ita*) that span several characters. Attention can focus on the name's opening syllable to maintain consistency at the ending, something a simple recurrent state struggles to propagate.
- The model generates 75.5% of names with valid Indian suffixes, compared to 77.9% for vanilla RNN — essentially equal, but with slightly higher novelty (0.136 vs. 0.114), showing better generalization.

14. Failure Modes

Model	Repeated Chars	Unnatural Clusters	Truncated ($\leq$2)	Too Long (>12)	Memorised
Vanilla RNN	0	6	0	0	886
BiLSTM	0	17	1	0	3
RNN + Attention	0	8	0	0	864

The dominant failure mode for Vanilla RNN and Attention is **memorisation**: with only 1,000 training names and strong pattern learning, both models heavily reproduce training examples ($\sim$86% of generated names). This reflects overfitting rather than creative generalisation.

BiLSTM's primary failure is **unnatural consonant clusters** (e.g. *Pafnswej*, *Sensjwil*) and phonetically incoherent sequences (e.g. *Amvewef*, *Gefnuded*). These arise from the train/generate distribution shift: the model is generating from a degraded internal state that does not correspond to any learned phonetic pattern.

Both Vanilla RNN and Attention produce the expected Indian suffix patterns (*-a*, *-ya*, *-an*, *-ish*, *-ka*, *-ti*) at high rates, confirming that phonotactic structure is correctly captured.

15. Key Insights — Name Generation

1. **High novelty $\neq$ high quality.** BiLSTM's near-perfect novelty score (0.997) is a cautionary result: it signals that the model has diverged from the training distribution, not that it has learned creative generalisation.
2. **BiLSTM is architecturally unsuited for open-ended generation.** Bidirectional training inherently conditions on both past and future context, but autoregressive generation is strictly left-to-right. This mismatch renders BiLSTM unreliable for character-level name generation without architectural modifications (e.g., decoder-only LSTM or schedule sampling).
3. **Attention provides the best quality/novelty trade-off.** With 75.5% valid Indian suffixes and 13.6% novel names, the Attention model demonstrates that soft memory of earlier hidden states improves phonotactic consistency across the full name length.
4. **Small datasets cause memorisation.** With 1,000 training names, both Vanilla RNN (886 memorised) and Attention (864 memorised) heavily reproduce the training set. A dataset of 5,000–10,000 names would likely push novelty rates significantly higher while maintaining realism.
5. **Parameter count alone does not predict generation quality.** BiLSTM has 6$\times$ more parameters than Vanilla RNN yet produces qualitatively worse output. Architectural compatibility with the task is more important than raw capacity.
6. **Quantitative and qualitative evaluation are both essential.** No single metric captures generation quality. Novelty, diversity, suffix adherence rates, and human inspection together provide a complete picture.

Conclusion

Conclusion

This report presented solutions to both problems in CSL 7640 Assignment 2.

In **Problem 1**, Word2Vec models (CBOW and Skip-Gram with Negative Sampling) were trained on a 47-document, 18,922-token corpus scraped from IIT Jodhpur's online sources. The central finding is that Skip-Gram with dim=300, window=2, and 5 negative samples produces the sharpest embeddings for this domain-specific corpus, with cosine similarities to semantically related words consistently 10–15% higher than CBOW. Visualisations confirm that both models capture meaningful semantic clusters (degree programs, research, campus life), but the small corpus size limits the accuracy of analogy-solving. *Corpus size is the dominant factor limiting embedding quality*: even doubling the token count would likely yield substantially more coherent analogy results.

In **Problem 2**, three character-level sequence models were trained on 1,000 Indian names. The key learning outcome is that *architectural compatibility with the generation procedure matters more than model capacity.* BiLSTM, despite its large parameter count and low training loss, produces nonsensical names due to the bidirectional train/generate mismatch. RNN+Attention achieves the best qualitative results by using a soft attention mechanism that preserves phonetic context without any directional inconsistency. Both RNN and Attention models are limited by memorisation, underscoring that *dataset size is the primary bottleneck* in small-scale generative tasks.

Taken together, the two problems reinforce a consistent theme: the quality of any NLP model is fundamentally bounded by the size and diversity of its training data, and architectural choices must be informed by how the model will be used at inference time, not just how it is trained.